

Universitat d'Andorra (UdA)

TREBALL VIRTUAL I

Ampliació de tecnologies aplicades

Bàtxelor en informàtica

Autors:

Pau Ivan Santos Villalba

Eric Torrontera Ruiz

TAULA DE CONTINGUTS

1. Introducció i visió general del projecte GameHub.....	4
1.1 Resum de les tecnologies aplicades.....	4
1.2 Capítols del llibre que hem aplicat.....	5
1.3 Funcionalitats que finalment hem implementat.	5
2. Arquitectura i models de dades que hem emparat	6
2.1 Arquitectura General	6
2.2 Serveis de Docker Compose.....	7
2.3 Diagrama de flux de la nostra API REST	7
2.3 Model de dades de la nostra web (1FN)	8
2.4 Diagrama de seqüència del sistema.....	9
3. Guia de desplegament amb Docker Desktop	10
2.1 Resum del docker-compose.yml.....	11
2.2 Resum del Dockerfile	11
3.3 Gestió de fitxers multimèdia (imatges).....	12
4. Documentació de la nostra API REST	13
4.1 Descripció dels endpoints	13
4.2 Proposta de consum en Unity.....	13
5. Resum dels requeriments del TV1 que hem implementat	14
5.1 Integrar funcionalitats de com a mínim dues de les següents categories:.....	14
5.2 Utilitzar mínim dues llibreries externes tractades al llibre, com ara:	14
5.3 Incloure una API pròpia (RESTful)	15
5.4 Aplicar plantilles HTML amb estilització coherent (es pot usar Bootstrap o Tailwind CSS)	15
5.5 Desplegar la versió final en un entorn real o de simulació.....	15
6. Validació del sistema i demostració visual.....	16
6.1 Posada en marxa amb docker (Port 8000).....	16
6.2 Creació i login de admin. (8000/admin).....	17
6.3 Creem un nou post DevLog.....	18
6.4 Creem un usuari normal i fem login.	19
6.5 Comentem un post de DevLog amb un usuari normal.	20

6.6 Creem un post nou a l'apartat de blogs com usuari normal	20
6.7 Testing des endpoints de la nostra API RESTful.....	21

1. Introducció i visió general del projecte GameHub

GameHub és la plataforma web oficial per al nostre videojoc del TFG. Té com a objectiu centralitzar la comunicació del desenvolupament del joc i fomentar la participació de la comunitat de jugadors. La nostra plataforma ofereix:

- **Devlog CMS:** Un blog de desenvolupament on l'equip publica actualitzacions, notes de versió i anuncis.
- **Comunitat Social:** Un espai on els usuaris poden publicar contingut, respondre's entre ells i adjuntar imatges.
- **Botiga Virtual:** Una secció on els usuaris poden "comprar" skins per al personatge i moneda virtual del joc (Diamants i Monedes).
- **API REST:** Una API pública per exposar dades del devlog, útil per a futures integracions amb el client del joc.

1.1 Resum de les tecnologies aplicades

En aquest apartat hem resumit el conjunt d'eines i frameworks seleccionats per desenvolupar la nostra web i que hem mirat d'entendre:

- **Backend web:** Django 5.2
- **Frontend responsiu:** Bootstrap 5.3
- **Gestió d'imatges:** Pillow - *Capítol 4 (Social Website – Profile photo)*
- **Etiquetes:** django-taggit - *Capítol 2 (Blog Application – Tags)*
- **API REST:** Django REST Framework - *Capítol 15 (eLearning Platform API)*
- **Renderització Markdown:** python-markdown - *Capítol 3 (Blog – RSS Feed amb MD)*
- **Desplegament:** Docker Desktop + Docker Compose
- **Base de dades:** SQLite (desenvolupament)

1.2 Capítols del llibre que hem aplicat

Per realitzar aquest projecte hem utilitzat el codi proporcionat pel llibre "Django 5 By Example" (5a ed.) d'Antonio Melé. A continuació mostrarem una breu taula resum:

Capítol	Tema	Aplicació al nostre projecte
Cap. 1 - 3	Blog Application	Devlog CMS amb posts, tags, comentaris, feeds, cerca. Lo del markdown etc.
Cap. 4	Social Website	Autenticació, registre, perfils d'usuari
Cap. 8	Ecommerce	Botiga (skins + currency), cistella, comandes
Cap. 15	eLearning API	API REST amb DRF (viewsets, routers, serializers)

1.3 Funcionalitats que finalment hem implementat.

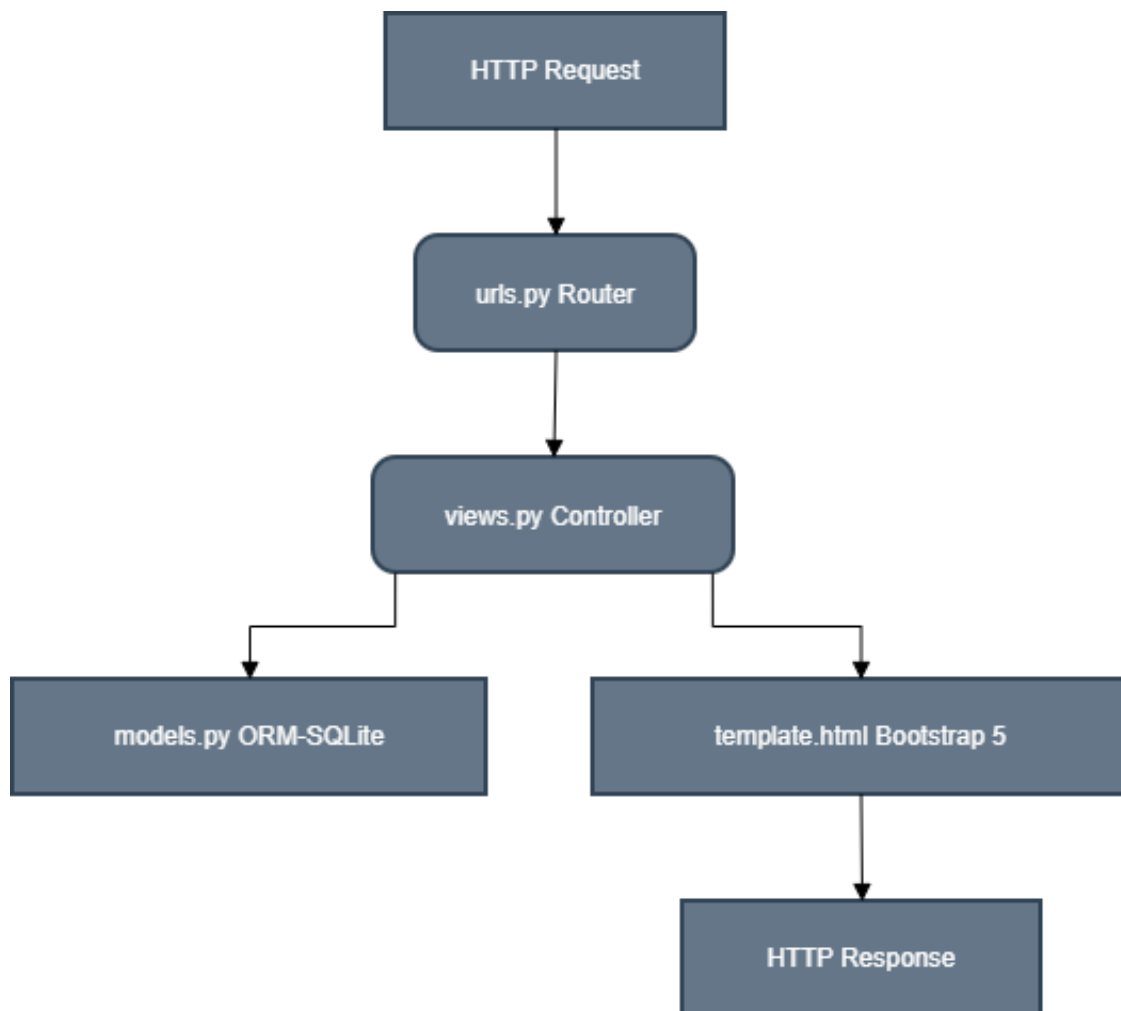
La nostra web del TFG integra un sistema de gestió de continguts amb paginació, filtratge per etiquetes i fils RSS per al seguiment del desenvolupament, complementat amb un mòdul d'autenticació que permet la gestió de perfils i l'accés a una xarxa social interactiva. Així mateix, hem implementa una botiga virtual amb carret de la compra gestionat per sessions. Finalment comentar que hem incorporat una API REST de consulta que facilita la interoperabilitat de les dades mitjançant punts de connexió read-only. Aquesta API serà utilitzada pel nostre videojoc a Unity per tal de traslladar el devlog a una secció de notícies.

2. Arquitectura i models de dades que hem emparat

En aquest bloc definirem l'estructura lògica i l'organització de la persistència del projecte, fonamentada en un esquema de dades en primera forma normal que garanteixi la integritat i l'escalabilitat del sistema.

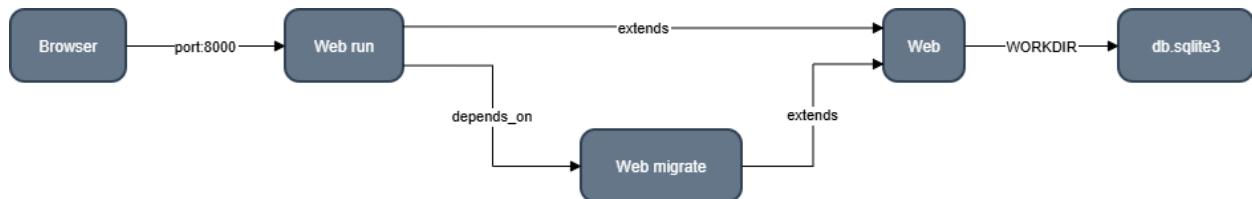
2.1 Arquitectura General

Com hem vist a classe, el projecte s'articula mitjançant el patró de disseny Model-View-Template, el qual permet un desacoblament eficaç entre la lògica de negoci, la gestió de dades i la presentació de la interfície d'usuari.



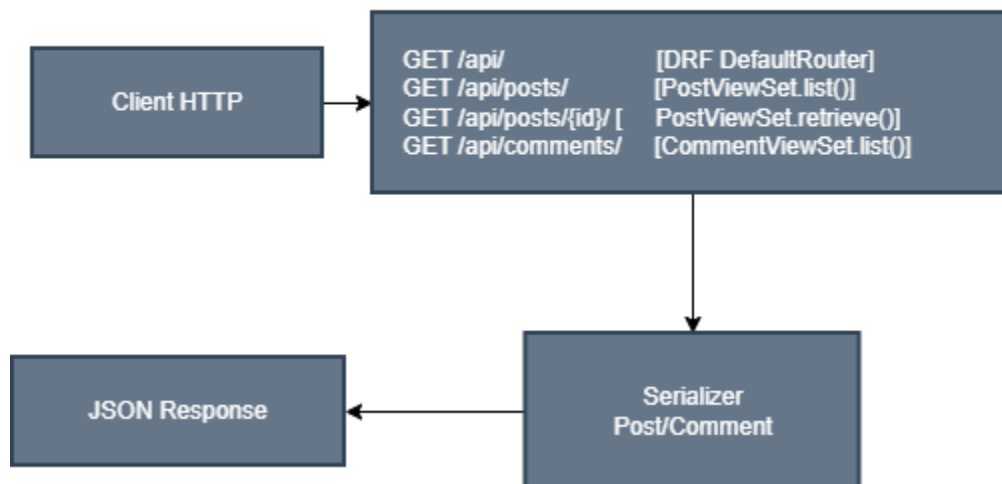
2.2 Serveis de Docker Compose

Per tal de mantindre el nostre entorn de treball net i desplegar la nostra proposta de web hem utilitzar Docker Compose amb Docker Desktop.



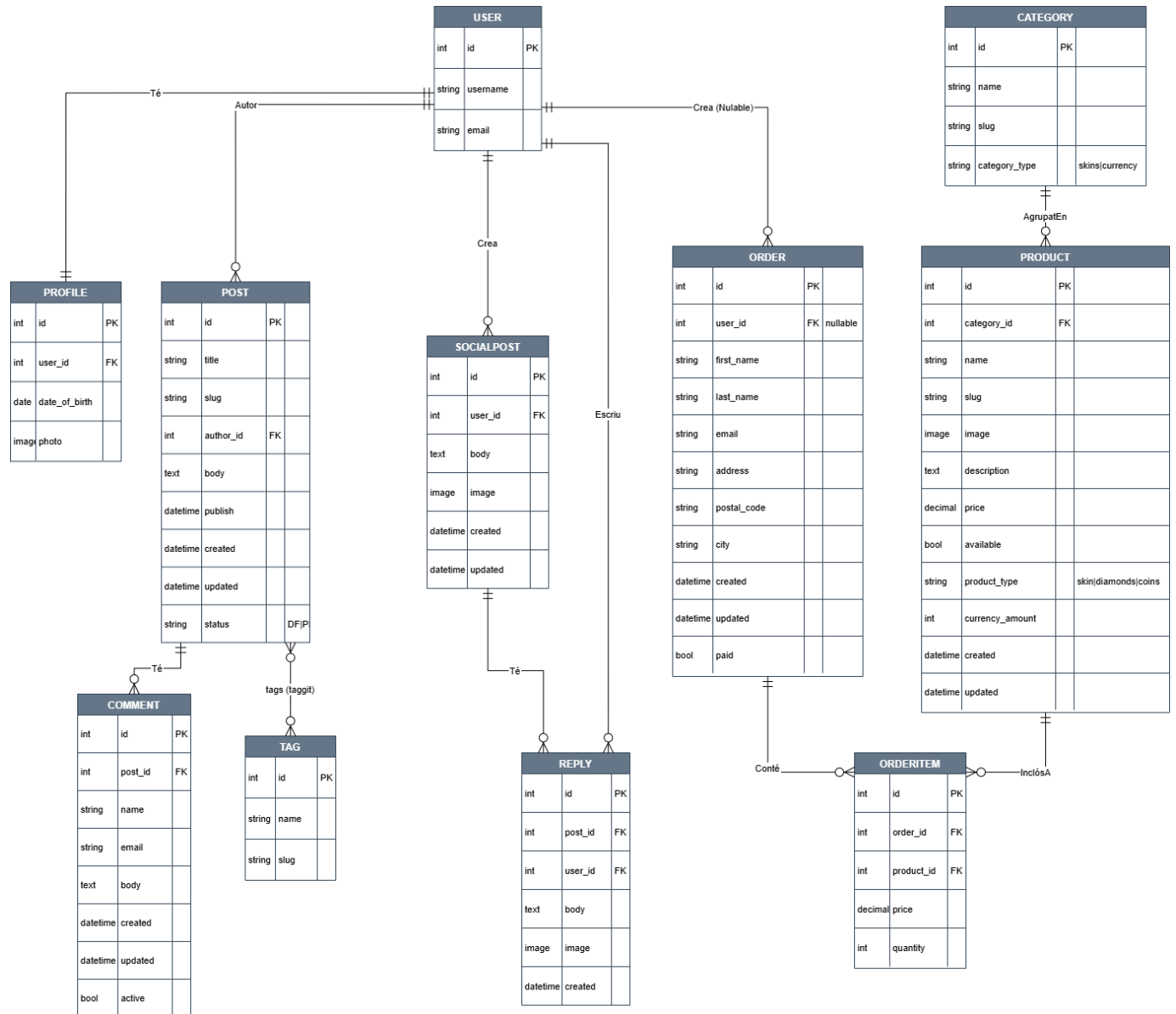
Nota: Als exemples del llibre hem vist que utilitza **do.sh** amb el resums de les comandes. Nosaltres com estem familiaritzats amb docker compose utilitzem les comandes normals, tot i que el **do.sh** és útil quan les comandes es fan llargues.

2.3 Diagrama de flux de la nostra API REST



Nota sobre el format de dades: Tot i que el format d'intercanvi principal i el que consumeix el client de Unity és JSON, la implementació aprofita la Interfície Navegable (Browsable API) de Django REST Framework

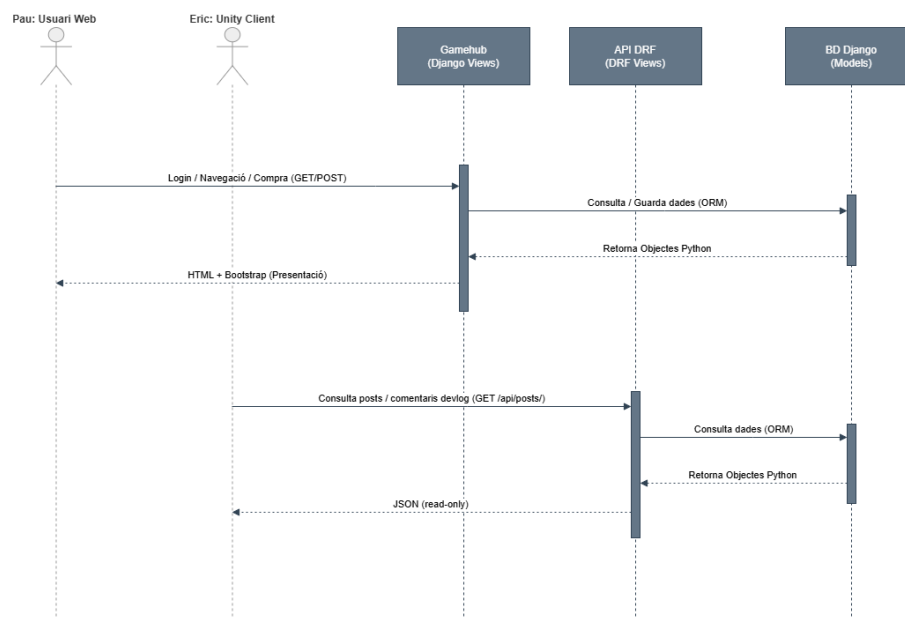
2.3 Model de dades de la nostra web (1FN)



2.4 Diagrama de seqüència del sistema

Hem considerat interessant afegir la dinàmica d'interacció entre els diferents components de l'arquitectura del projecte. El sistema s'ha dissenyat sota un enfocament multiclient, on un únic backend centralitzat en Django gestiona la lògica de negoci i l'emmagatzematge de dades, servint simultàniament a dues plataformes diferents:

- A. **Interfície Web (Usuari):** Representa el flux convencional on l'usuari interactua mitjançant un navegador. El servidor processa les peticions a través de les views de Django i retorna respostes en format HTML estilitzades amb el framework Bootstrap 5. Aquest flux permet la gestió de perfils, la navegació pel contingut i la realització de compres.
- B. **Integració amb el motor de joc (Unity):** Descriu com el client de joc consumeix dades de forma externa. En aquest cas, les consultes es dirigeixen a l'API DRF (Django REST Framework), la qual actua com una capa d'abstracció que transforma la informació de la base de dades en paquets de dades JSON de només lectura.



Aquesta estructura garanteix la integritat de les dades, ja que tant el portal web com el videojoc operen sobre la mateixa base de dades SQLite a través de l'ORM de Django, assegurant que qualsevol canvi en el devlog o en l'estat dels productes es reflecteixi automàticament en ambdós clients.

3. Guia de desplegament amb Docker Desktop

En aquest apartat s'estableix el procediment operatiu per a la posada en marxa de l'entorn de producció i desenvolupament mitjançant la virtualització de contenidors.

El procés d'inicialització comença amb la construcció de la imatge del projecte mitjançant la comanda “docker compose build”. Aquesta fase executa les instruccions definides al *Dockerfile*, basant-se en la imatge oficial python:3.12.3-slim per configurar l'entorn de treball i transferir el codi font al directori /code del contenidor.

Un cop configurat l'entorn, la gestió de la persistència de dades es realitza a través de les migracions de Django. Tot i que el servei web_migrate del fitxer de configuració automatitza aquest procés en iniciar el sistema, si volem es possible executar-les de manera manual mitjançant “docker compose run --rm web python manage.py migrate” o també amb la comanda que ve als exemples del llibre de do.sh. De la mateixa manera, la creació del superusuari administrador es gestiona de forma interactiva amb la comanda createsuperuser per garantir l'accés al panell de control de la plataforma.

Finalment, l'execució del servidor es realitza en segon pla mitjançant “docker compose up -d web_run”, habilitant l'accés als diferents mòduls de la plataforma a través del port local 8000, incloent-hi la interfície pública, el panell d'administració i els *endpoints* de l'API REST. Per finalitzar la sessió de treball i l'execució dels contenidors de manera controlada, s'utilitza la comanda “docker compose down”.

1. *docker compose build*
 2. *docker compose run --rm web python manage.py migrate*
 3. *docker compose run --rm web python manage.py createsuperuser*
 4. *docker compose up -d web_run*
 5. *Docker compose down*
-
- A. Web principal: *http://localhost:8000*
 - B. Panell d'admin: *http://localhost:8000/admin/*
 - C. API REST: *http://localhost:8000/api/*

Nota: Als exemples del llibre hem vist que utilitza **do.sh** amb el resums de les comandes. Nosaltres com estem familiaritzats amb docker compose utilitzem les comandes normals, tot i que hem vist que el do.sh és útil quan les comandes es fan llargues.

2.1 Resum del docker-compose.yml

```
services:
  web:
    # Imatge base (no s'inicia directament)
    build: .
    # Usa el Dockerfile al directori actual
    volumes:
      - ./code
        # Munta el codi local al contenidor

  web_migrate:
    # Servei per aplicar migracions
    extends:
      service: web
    # Hereta configuració de 'web'
    command: python /code/manage.py migrate

  web_run:
    # Servidor de desenvolupament Django
    extends:
      service: web
    command: python /code/manage.py runserver 0.0.0.0:8000
    ports:
      - "8000:8000"
    # Mapeja port contenidor -> host
    depends_on:
      - web_migrate
    # Espera que les migracions s'apliquin
```

Nota: El volum - ./code significa que qualsevol canvi al codi local es reflecteix immediatament al contenidor (Amb això volem dir que funciona sense necessitat de rebuild).

2.2 Resum del Dockerfile

```
FROM python:3.12.3-slim
# Imatge mínima de Python

RUN apt-get update -qq \
    && rm -rf /var/lib/apt/lists/*
# Neteja cache apt

ENV PIP_DISABLE_PIP_VERSION_CHECK 1
# No comprova versions de pip
ENV PYTHONDONTWRITEBYTECODE 1
# No crea fitxers .pyc
ENV PYTHONUNBUFFERED 1
# Output en temps real

WORKDIR /code
# Directori de treball

COPY requirements.txt .
RUN python -m pip install -r requirements.txt
# Instal·la deps

COPY . .
# Copia el codi
```

3.3 Gestió de fitxers multimèdia (imatges)

Les imatges pujades pels usuaris es desen localment dins `media/`, separades per funcionalitat i data (%Y/%m/%d). Aquesta estructura facilita l'organització, evita directoris massa grans i millora el manteniment/còpies de seguretat.

Fotos de perfil: `users/%Y/%m/%d/`

Imatges socials: `social/%Y/%m/%d/` (i també respostes a `social/replies/%Y/%m/%d/`)

Imatges de botiga: `products/%Y/%m/%d`

A nivell de configuració, el projecte defineix `MEDIA_URL` i `MEDIA_ROOT`, i en desenvolupament les rutes de media es serveixen des de `urls.py` amb:

```
static(settings.MEDIA_URL, document_root =settings.MEDIA_ROOT)
```

A més, en usar `ImageField`, Django valida fitxers d'imatge amb Pillow.

Exemple de camp d'imatge que usa Pillow:

```
photo = models.ImageField(upload_to='users/%Y/%m/%d/', blank=True)
```

4. Documentació de la nostra API REST

Com hem comentat en apartats anteriors la nostra API REST exposa dades del Devlog perquè clients externs (Unity) puguin mostrar notícies i comentaris en una UI pròpia.

4.1 Descripció dels endpoints

L'intercanvi de dades s'organitza al voltant de dos recursos principals sota la base URL:

`http://localhost:8000/api/`

- A. **GET /api/posts/**: Exposa la totalitat de les entrades del blog de desenvolupament, incloent les etiquetes associades i un llistat niat dels comentaris actius.
- B. **GET /api/comments/**: Permet la consulta independent de les intervencions dels usuaris, vinculades sempre a una entrada existent del *devlog*.
- C. **GET /api/posts/{id}/**: Detall d'un post concret.
- D. **GET /api/comments/{id}/**: Detall d'un comentari concret.

Com que els endpoints són de **només lectura**, la consulta pública està permesa i no hi ha operacions de creació/edició/esborrat a l'API.

4.2 Proposta de consum en Unity

Per a la integració en el client de joc, es definiríem un flux de treball basat en classes de transferència de dades (DTO). Unity realitza peticions GET mitjançant UnityWebRequest, deseria l'objecte JSON resultant en estructures C# i trasllada la informació als components de la UI (com *ScrollViews* o *Text Elements*) per mostrar el feed de notícies actualitzat en temps real. Segurament si tenim temps ampliarem l'API per skins i monedes.

5. Resum dels requeriments del TV1 que hem implementat

Cobertura del nostre projecte:

- [✓] Devlog/CMS (carpeta devlog)
- [✓] Autenticació i perfils (carpeta account)
- [✓] Interacció social (carpeta social)
- [✓] E-commerce (carpetes shop, cart i orders)
- [✓] API REST (carpeta api)

Link al nostre repositori públic de GitHub: <https://github.com/Andormix>

5.1 Integrar funcionalitats de com a mínim dues de les següents categories:

- [✓] Gestió de continguts (Blog)
- [✓] Autenticació i perfils d'usuari
- [✓] Interacció social (comentaris)
- [✓] Comerç electrònic (catàleg, carret, ~~passarel·la de pagament~~)
- [✓] Sistemes de cerca o recomanació
- [X] Funcionalitats asíncrones (amb WebSockets, Celery o AJAX)
- [✓] REST API (amb Django REST Framework)

5.2 Utilitzar mínim dues llibreries externes tractades al llibre, com ara:

- [✓] django-rest-framework
- [] easy-thumbnails
- [X] django-redis
- [X] celery o stripe
- [✓] Pillow
- [X] requests
- [✓] django-taggit (addicional)

Nota: Hem investigat una mica la llibreria de Python utilitzada per convertir la sintaxi Markdown dels posts en codi HTML apte per a lectors de RSS Feed. La del Capítol 3. Requirements.txt: inclou djangoestframework, Pillow i django-taggit

5.3 Incloure una API pròpia (RESTful)

[✓] Implementat API REST que exposa dades del Devlog perquè clients externs (Unity) puguin mostrar notícies i comentaris en una UI pròpia.

- GET /api/posts/
- GET /api/posts/{id}/
- GET /api/comments/
- GET /api/comments/{id}/

Hem investigat una mica que és ful amb les sis restriccions definides per Roy Fielding (el creador del terme).

5.4 Aplicar plantilles HTML amb estilització coherent (es pot usar Bootstrap o Tailwind CSS)

[✓] Hem reutilitzat les plantilles dels capítols i actualitzat el seu estil per tal de que sigui semblant a la UI del nostre videojoc. Com a millora podríem fer un arxiu Style centralitzat apart del base.html. Ara mateix esta una mica “hardcoded”. El header i footer compartit parteixen de la carpeta templates com al exemple del llibre, es el mateix, adaptat. Concretament esta implementat amb Bootstrap 5.

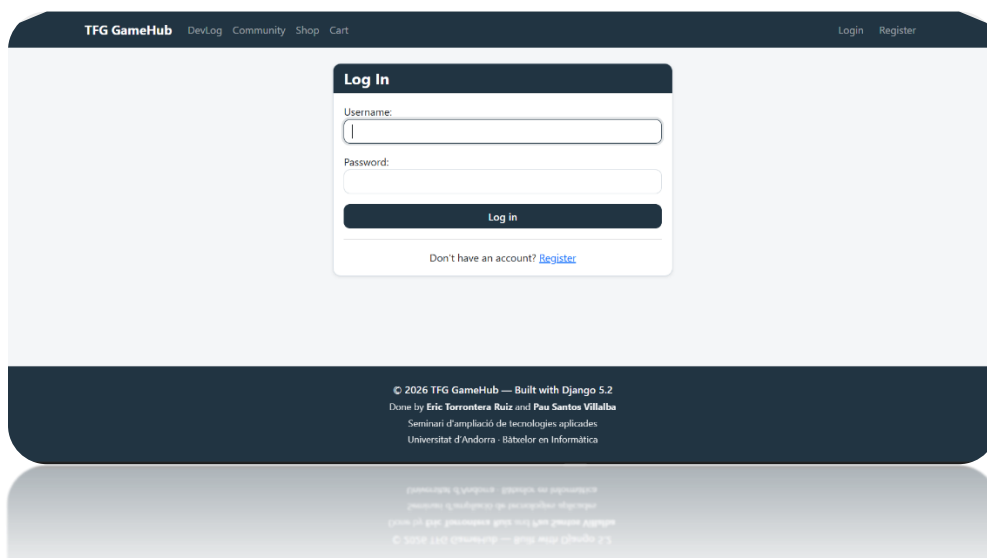
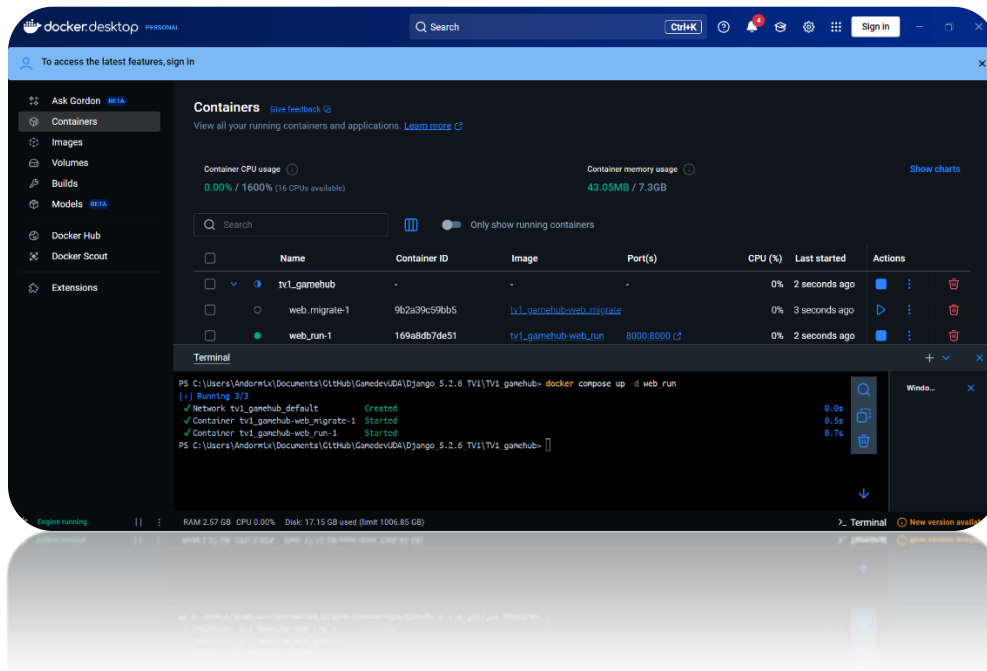
5.5 Desplegar la versió final en un entorn real o de simulació.

[✓] Hem validat el projecte en simulació amb Docker Desktop + Docker Compose. Com a evolució cap a entorn real, hem preparat desplegament de producció amb variables d'entorn (DEBUG=False, ALLOWED_HOSTS, SECRET_KEY), servidor WSGI (Gunicorn), base de dades PostgreSQL i persistència de fitxers media/ i static. Aquesta configuració és desplegable en un proveïdor cloud compatible amb Docker.

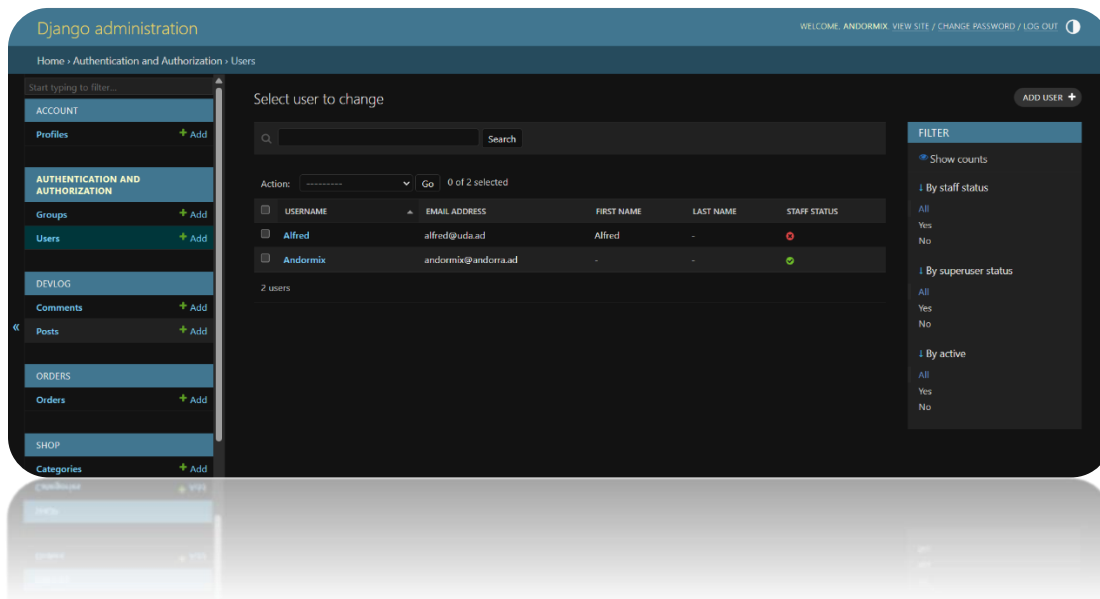
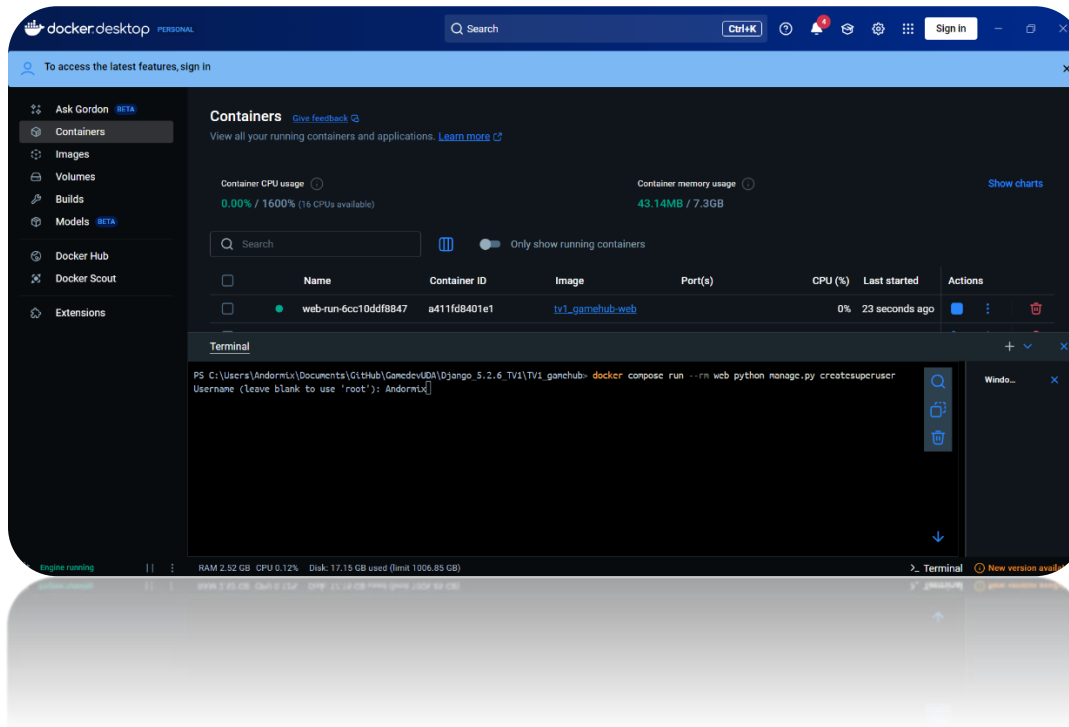
6. Validació del sistema i demostració visual

En aquest apartat ens deixem de teories i anem a la pràctica: ensenyem amb captures de pantalla que el sistema funciona de veritat.

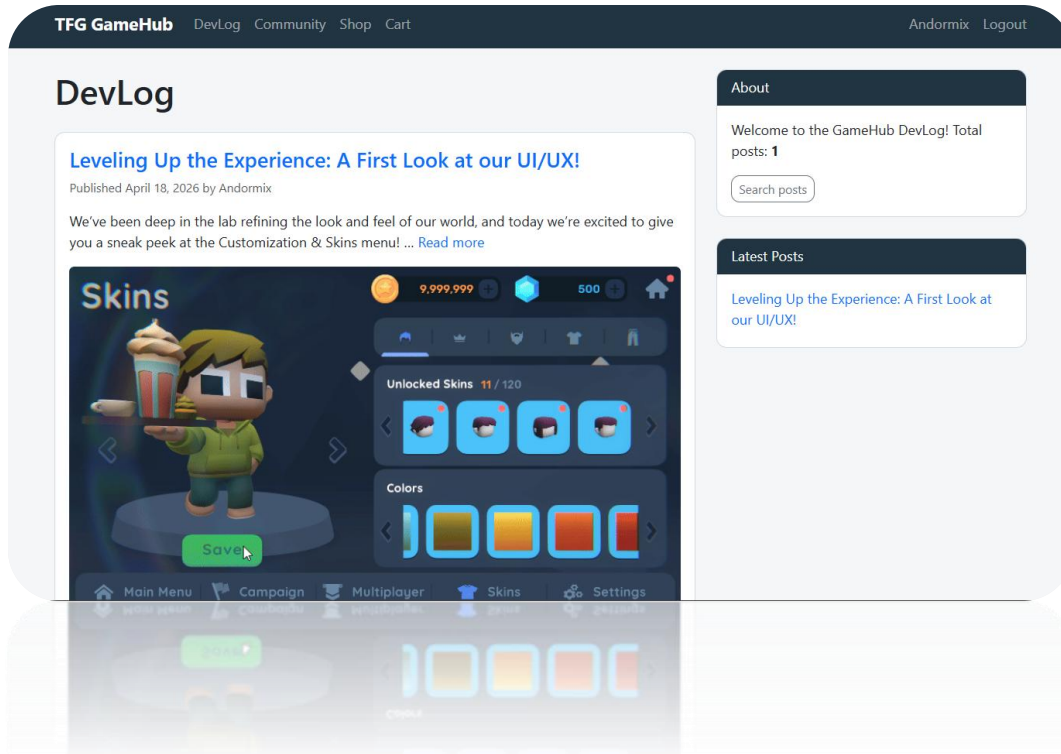
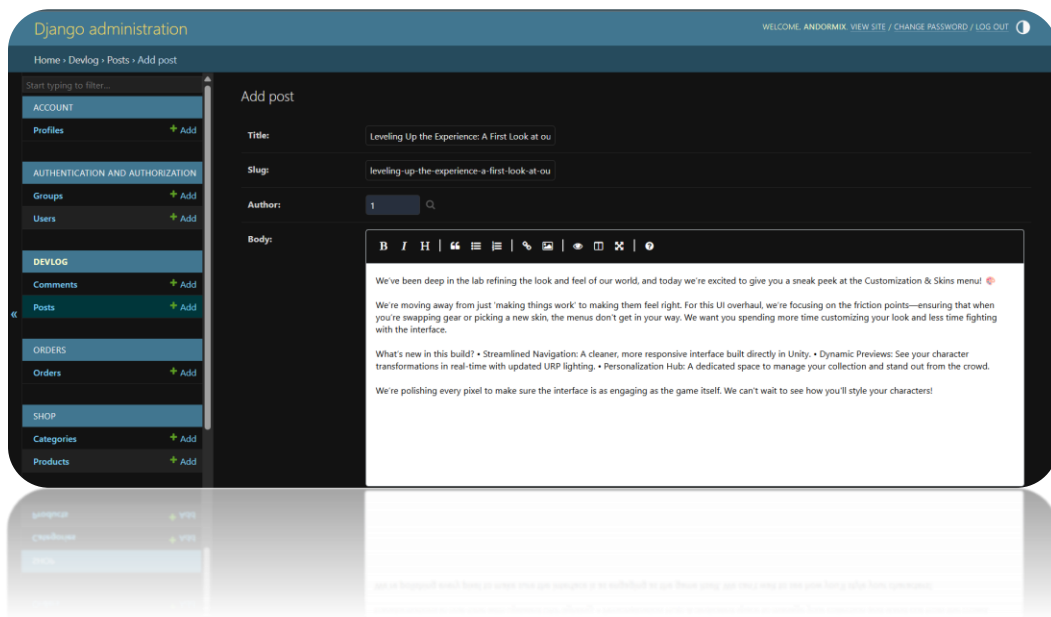
6.1 Posada en marxa amb docker (Port 8000)



6.2 Creació i login de admin. (8000/admin)



6.3 Creem un nou post DevLog.



6.4 Creem un usuari normal i fem login.

TFG GameHub

DevLogCommunityShopCart

LoginRegister

Create Account

Username

Choose a username

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

First name

Your first name

Email address

you@example.com

Password

Create a password

Repeat password

Repeat your password

Register

TFG GameHub

DevLogCommunityShopCart

AlfredLogout

Dashboard

Welcome back, **Alfred!**

Edit Profile

Change Password

New Community Post

Visit Shop

Account Info

Username:

Alfred

Email:

alfred@uda.ad

Joined:

April 18, 2026

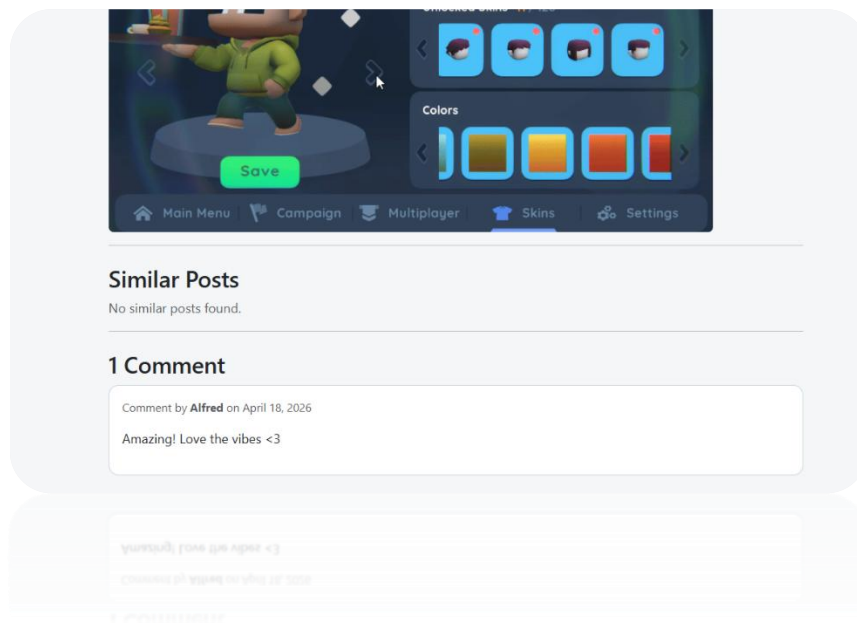
© 2026 TFG GameHub — Built with Django 5.2

Done by **Eric Torrontera Ruiz** and **Pau Santos Villalba**

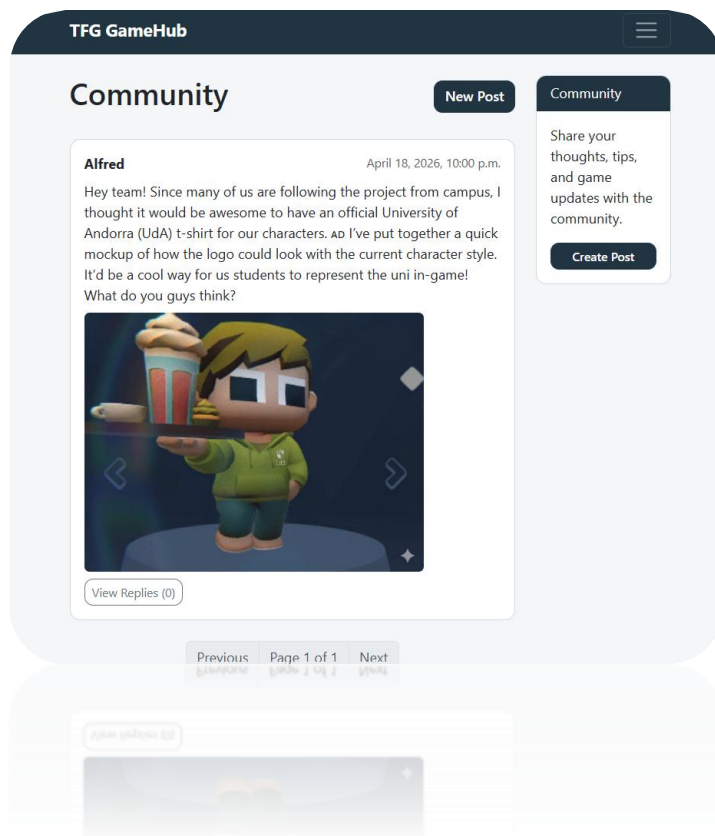
Seminari d'ampliació de tecnologies aplicades

Universitat d'Andorra - Bàtzelor en Informàtica

6.5 Comentem un post de DevLog amb un usuari normal.

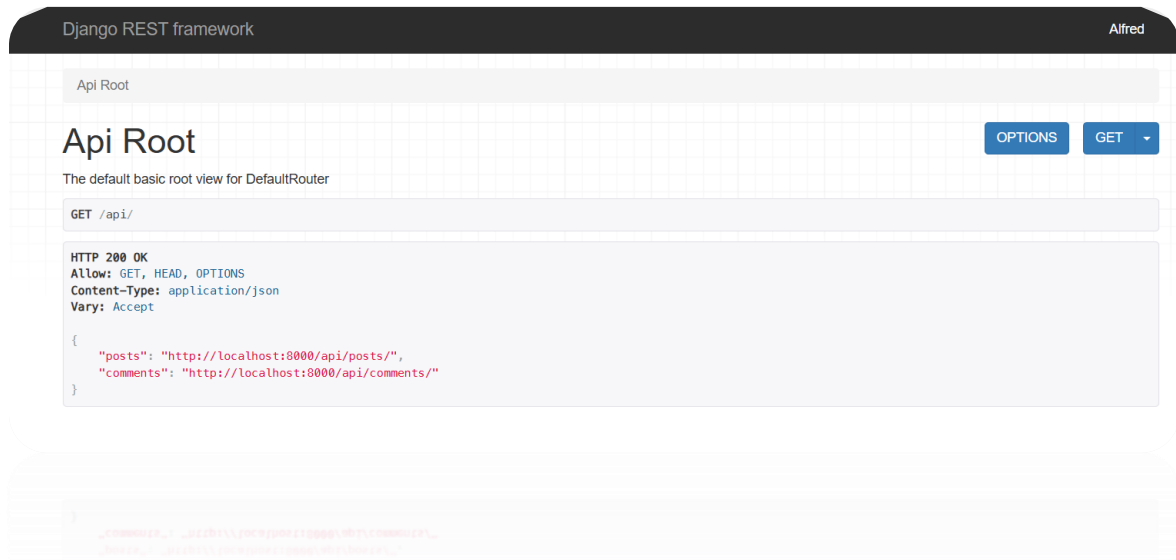


6.6 Creem un post nou a l'apartat de blogs com usuari normal

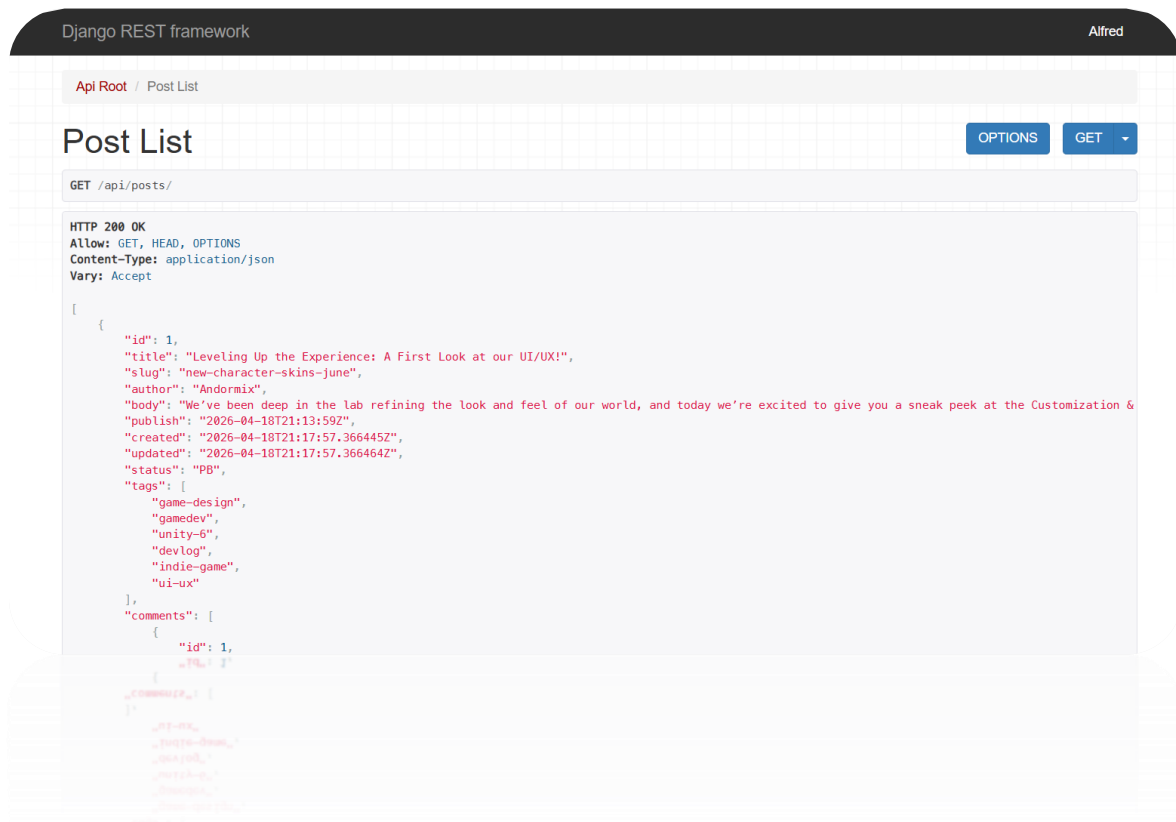


6.7 Testing des endpoints de la nostra API RESTful.

A. <http://localhost:8000/api/>



B. <http://localhost:8000/api/posts/>



C. <http://localhost:8000/api/posts/1/>

Django REST framework Alfred

Api Root / Post List / Post Instance

Post Instance

OPTIONS GET

GET /api/posts/1/

HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```
{
  "id": 1,
  "title": "Leveling Up the Experience: A First Look at our UI/UX!",
  "slug": "new-character-skins-june",
  "author": "Andormix",
  "body": "We've been deep in the lab refining the look and feel of our world, and today we're excited to give you a sneak peek at the Customization & Skin",
  "publish": "2026-04-18T21:13:59Z",
  "created": "2026-04-18T21:17:57.366445Z",
  "updated": "2026-04-18T21:17:57.366445Z",
  "status": "PB",
  "tags": [
    "game-design",
    "gamedev",
    "unity-6",
    "devlog",
    "indie-game",
    "ui-ux"
  ],
  "comments": [
    {
      "id": 1,
      "post": 1,
      "name": "Alfred",
      "body": "Amazing! Love the vibes <3",
      "created": "2026-04-18T21:52:38.866745Z"
    }
  ]
}
```

D. <http://localhost:8000/api/comments/>

Django REST framework Alfred

Api Root / Comment List

Comment List

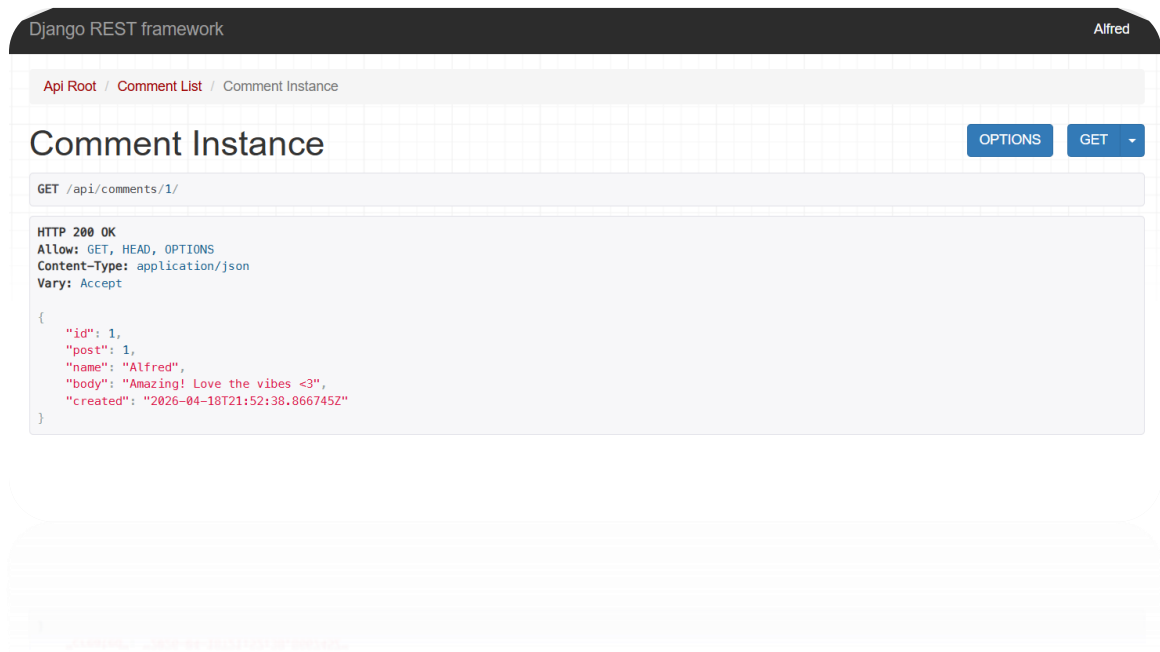
OPTIONS GET

GET /api/comments/

HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```
[
  {
    "id": 1,
    "post": 1,
    "name": "Alfred",
    "body": "Amazing! Love the vibes <3",
    "created": "2026-04-18T21:52:38.866745Z"
  }
]
```

E. <http://localhost:8000/api/comments/1/>



Amb aquesta bateria de tests caixa negra visuals queda demostrat que el flux de dades entre el *backend* i els diferents usuaris funciona sense problemes. Hem validat des de la posada en marxa amb Docker, que ens assegura un entorn estable, fins a la interacció real entre administradors i usuaris normals. El fet que els comentaris i els posts apareguin correctament tant a la web com als *endpoints* de l'API confirma que l'arquitectura del projecte és funcional, integrada i que està a punt per ser consumida pel client de joc o qualsevol plataforma externa.

Bibliografia

- [1]. Alcón Beltran, A. (2025). *Seminari Ampliació de tecnologies aplicades* [Apunts de classe]. Campus Virtual de la Universitat d'Andorra. <https://virtual.uda.ad/>
- [2]. Melé, A. (2024). *Django 5 By Example* (5a ed.). Packt Publishing.